* Chocolate distribution Problem GFG

arr = {7, 3, 2, 4, 9, 12, 56}, m = 3

* Sort = {2, 3, 4, 7, 9, 12, 56}

m = 3

j = m - 1

②

~~i++~~

i+②

$M_x - M_n$

4 - 2 = ② Ans

7 - 3 = 4

9 - 4 = 5

12 - 7 = 5

56 - 9 → Too large

(m =) 1

min

(INT MAX, 2)

(2, 4) 2

7 - 3 = 4

* Minimum Cost of Ropes:

arr = { 4, 3, 2, 6 } o/p = 29

O(1)

nlogn ① Sort {2, 3, 4, 6} Cost = 5

+

nlogn ⑪ Sort {5, 4, 6} Cost = 9

{4, 5, 6}

+

{9, 6} Cost = 15

nlogn Sort {6, 9}

⑮

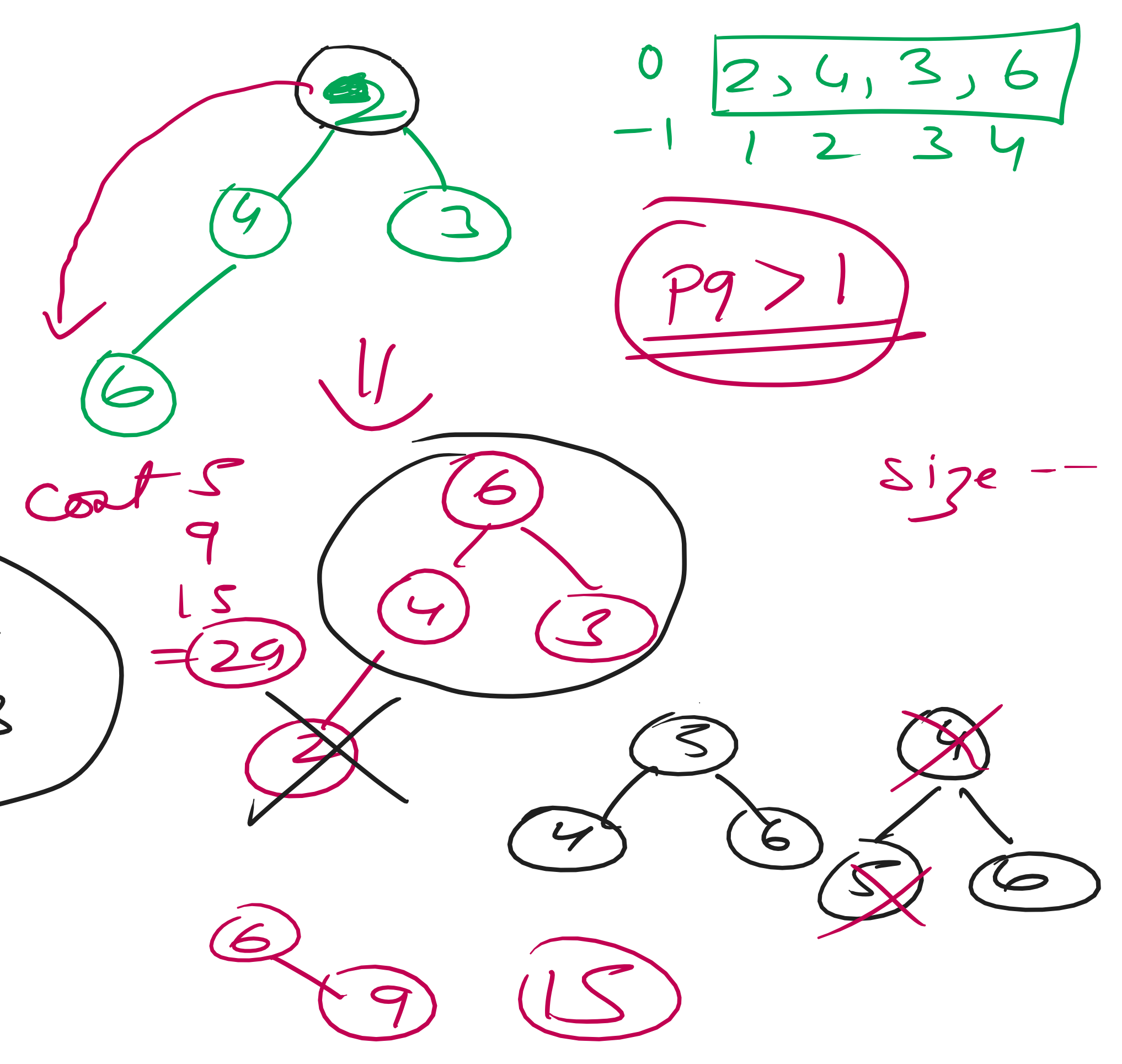
Too large

㉙

(4, 3, 2, 6)

min → heap

(priority queue)



CBT

0 | 2, 4, 3, 6

-1 | 1 2 3 4

pq > 1

size --

cost 5

9

15

= ㉙

first = 2

second = 3

2+3 = ⑤

f = 4 = 9

s = 5

Fractional Knapsack Problem (Greedy)

N - items

wt → [ ]

val → [ ]

Knapsack (Bag)

Capacity → W

(50)

50

- 30

20kg

N = 3, W = 50

Maximize val → [60, 100, 120]

kg wt → [⑩, 20, ㉚] = 60

1st 6 2nd 5 3rd 4

$\left[\frac{V}{wt}\right]$

Fraction

1st $\frac{60}{10} = 6$

2nd $\frac{100}{20} = 5$

3rd $\frac{120}{30} = 4$

per unit value

max:

6, 5, 4

10kg + 20kg + 20kg

× 6 × 5 × 4

60 + 100 + 80

= (240)

20 × 5 30 × 4

120

100 + 120

= (220)

W = (50) =

W = 0

* Calculate the per unit value:

vec<pair<double, Item>> V;

puv

$\left(\frac{V}{wt}\right)$

puv Item Item

(6, 60, 10), (5, 100, 20), (4, 120, 30)

first second W

V[i]. W   6 × W

Nikunj & Donuts :→

[3], [1, 3, 2] ⇒ Sort = desc

c_i c_{i+1} c_{i+2}

0 1 2

j = 0

= 1

= 2

3

2^0

1 × ③ = 3

2 × 3 = 6

4 × 3 = 12

calories = 1 : $2^j \times k$

k = 1

$2^0 \times 1 = 1 \times 1 = 1$ miles

calories = 3 : $2^j \times k$

$2^1 \times 3 = 2 \times 3 = 6$ miles

calories = 2 : $2^j \times k$

$2^2 \times 2 = 4 \times 2 = 8$ miles

min ⑮ miles

[3, 2, 1] j = 0, 1, 2

k = 3, $2^j \times k$

$2^0 \times 3$

1 × 3 = ③

+

k = 2, $2^j \times k$

$2^1 \times 2$

2 × 2 = ④ = ⑪

+

k = 1, $2^j \times k$

= $2^2 \times 1$

= 4 × 1 = ④

Policeman & Thieves + Huffman Encoding

(1:00 - 3:00)

(DT + dp)